

Insufficient Logging and Monitoring

Esteban Barbosa

¹ Princeton University, Princeton NJ 08544, USA

² Springer Heidelberg, Tiergartenstr. 17, 69121 Heidelberg, Germany
lncs@springer.com

Abstract. This paper presents what logging is, the types there are, where they are stored, what is a good log, and what is a bad log. It will also present what monitoring is, what good monitoring is, and what bad monitoring is. It will also briefly touch observability and the difference between it and monitoring. We will also show a code demo with ruby on rails of a good and bad example of insufficient logging and monitoring.

1 Introduction

This project will show a website with a bad log structure, not able to give the security analyst the information in a concise manner. Bad clock, logs that not standard and instead use a variety of times to log the date and time of the event. Bad content, is where the log shows the users personal information unencrypted or using personal information as a unique ID. We will explain why this is considered insufficient and will demonstrate a website that has these flaws. Then we will show a website that has none of such problems, explain the fixes and why it was necessary to have those fixes.

1.1 What is logging and monitoring?

Logging is the practice of recording events and information, to use for security or as a timeline of events. Monitoring is a security practice of using the logs collected, compiling them, then analyzing them to notice trends then taking a response to abnormal events.

A real-world example of this would be a security camera, the recording, in this example, would be logging while monitoring is someone watching the recorded footage to look for suspicious activity.

Types of logs. In cyber security, according to [1] there are many logs recording all sorts of events that fall into two categories: operational use cases, and security use cases. Operational logs are events that happen in business such as transaction logs, used to help debug the application when error logs occur because a page doesn't load, monitor performance, such as how fast a page loads, and other requirements that a business have in their logging policy. For example, in [2] provides guidelines to have the logging system include the logging state, when it starts, stops, or pauses. Security logs are

events logged that have to do with keeping the application secure, these logs are used to mitigate, lessen the impact: damage that a vulnerability if exploited can do to a business, of certain security vulnerabilities such as broken access control. They are also used for digital forensics to track down what happened, who did it, and how to prevent the same attack from occurring again.

What are events? Events are actions that create an individual log. While there isn't a set industry wide format, according to [2], the events are supposed to answer 5 questions. When, what, who, where, and which. Event attributes are the individual information in the event that answers when, what, who, where, and which. For example, when a user signs into their account, the log created would be a list containing the users' unique ID, saying who they were. It would get their IP address, what page on the website they were on, the UR, showing where, whether it succeed or failed along with a tiny description of what happened such as user as signed in, showing what happened. The time and date it happened, showing when it happened, what resources got accessed, which would be the database containing the user's information to verify them.

How to monitor. Monitoring works by using an automated tool called SIEM, security information and event management, as the automated tool to collect, aggregate and analyze large volumes of logs and data. According to [4], SIEM works by collecting the raw data from applications, then normalize them to a standard format onto a centralized dashboard. SIEM is also used to respond to incidents by alerting IT, or security analysts about abnormal activities or take a preprogrammed response according to policy. Such as a user failing to log in 500 times in 0.5 seconds and the system blocking that IP address. This would be an example, according to [2] this type of monitoring is called anomalous behavior detection. The other types of monitoring are application health monitoring, monitors performance of the application, synthetic monitoring, mimics a user to test the users experience, and user monitoring, collecting user data to debug user-side performance issues.

Observability. According to [2] observability, is similar to monitoring but instead of checking the logs and responding to a threat like in monitoring. We want to measure the log attributes collected from all the users that use the site. Categorize the data and then analyze them as according to [5], using this information to discover patterns and trends in the application to discover weaknesses in the application and then develop responses to these new threats. Thus, we can log to collect data, respond to abnormal events with monitoring, and discover new flaws and bugs with observability.

1.2 Where are logs stored?

According to [1], logs are primary stored in 2 places, the file system on the local hard drive or a database system like SQL. They can also be stored in cloud services such as in [2] with AWS CloudWatch. There also should be backups of these logs stored on

another computer, in case the original logs gets deleted, tampered by a hacker, or corrupted.

Logs can be stored in all sorts of formats, from plain text files to JSON files, or raw binary, depending on where the log comes from. Businesses use SIEM that makes all logs of all types converted into a single standard format and then placed in a centralized dashboard instead of all over the system.

1.3 Log retention

According to [6], log retention, how long a log should be kept in the database before being deleted, varies from company to company. However, there are laws that require a log to be kept for a certain amount of time and disposed of in a certain way. A good standard for companies is to keep logs for at least one year before deleting the logs through secure overwrite or according to company policy.

1.4 What is insufficient logging and monitoring?

Insufficient logging and monitoring, according to [3], is ranked as 9th of the top 10 most critical web application security vulnerabilities. There are many ways logs can be insufficient, first is not having any logs. Second is incomplete log events, failing to answer who, what, when, where, or which. Third is having the user's personal information like their credit card number, unencrypted and visible with no security checks. Fourth is not having a backup of the logs or having them stored insecurely by having them unencrypted, or able to be accessed by users. Monitoring could be insufficient in firstly, just not checking the logs and having automated responses to break the kill chain, the kill chain being a series of steps the attacker does to cause damage to the application. Second, not having the system alert security analysts or IT that there is a problem.

How to test. There is not an easy way to test for insufficient logging and monitoring as this is a design and sometimes an implementation problem. Both of these are not code errors, if it is properly coded but it is insufficient because it shows all the logs in plain text, including usernames and passwords. A debugger won't tell you that there is a problem with the logs and monitoring system because the code is right. Instead, to test for this vulnerability, you have to review code by the company guideline and policy and according to best practices provided by OWASP.

Why it matters. Insufficient logging and monitoring can lead to attackers being unnoticed in the system, being able to progress further on the kill chain, the kill chain being a series of steps the attacker does to cause damage to the application. For a user on the platform they could steal their personal data, including personal bank details, and identity. For a business, they could lose money, lose future revenue, lose trust with their

users, and maybe face legal ramifications if their logging and monitoring system was willfully ignored.

1.5 Solution

The solution to insufficient logging and monitoring is to implement a logging and monitoring system, with a SIEM. According to [2], the events logged should be able to answer 5 questions, who, when, what, where, which. It should answer who, with a unique user ID, preferably not the user's personal information, and the users IP. It should be able to answer when, with a standard uniform time, including date, time, from when the event happened, and the date and time of when it was logged into the system. It should be able to answer what the type of event it is, a small description of the event, the severity of the event, low, medium, high. It should be able to answer where with application name and version, application address, the service the application is providing, the window that the event is on. And finally, which resources the event is affecting, their names, and number, whether objects like the user account got changed.

The SIEM should be able to alert security analysts or IT and respond to abnormal events in order to break kill chains when they are detected. The logs should also be reviewed by a security analyst with the proper tools and training make sure the logs are right and proper.

The logs should be stored in secure, encrypted folders, and that there is always a back-up that is verifying the original logs to see if it got tampered. Any user personal or sensitive data in the logs is encrypted.

The rest of this paper will demonstrate 2 websites, one with good logging and monitoring showing good structure, good clock, and good content. The other with bad logging and monitoring, bad structure, bad clock, and bad content.

2 Related Work

Logging - OWASP Cheat Sheet Series – The website provides guidance on building application logging mechanisms.

Best Practices for Event Logging and Threat Detection | CISA – The publication defines a baseline for event logging best practices to mitigate cyber threats.

Logging best practices - AWS Prescriptive Guidance – The website provides guidance on what logs are and what logs are useful.

SP 800-92, Guide to Computer Security Log Management | CSRC – This publication shows what logs are, what logs are useful. How organizations could create, maintain, and prioritize their log management.

AWS logging and monitoring guide for application owners | Amazon – This is a guide from amazon on their logging and monitoring guidelines

3 Design

For this project, I used ruby on rails, and github to store the code. With the good example being on the main branch, and the bad example being on the bad_example branch, Archer12222/secure-code-and-testing-project

Bad example:

```
GET / | Jump to view
def index
  @products = Product.all
  Rails.logger.info({
    # When - local time not UTC, no request_id
    timestamp: Time.now,
    # Who - logs PII (email)
    user: current_user&.email,
    # Where - no controller, action or method
    url: request.original_url,
    # What - vague,
    event: "Started",
    # Which - no ID
    resource: "products"
  }.to_json)
end
```

Fig. 1. Part of the web application for the index page, and the logic behind that page's event and log recording along with comments.

Good example:

```
Jump to view
def index
  @products = Product.all
  Rails.logger.info({
    # When
    timestamp: Time.now.utc,
    request_id: request.request_id,
    # Who
    user_id: current_user&.id,
    ip: request.remote_ip,
    # Where
    app: "RailsShop",
    controller: "store",
    action: "index",
    url: request.original_url,
    http_method: request.method,
    # What
    event: "store.index",
    severity: "info",
    result: "success",
    http_status: 200,
    # Which
    resource: "product",
    resource_count: @products.count
  }.to_json)
end
```

Fig. 2. Part of the web application for the index page, and the logic behind the page's event and log recording along with comments.

4 Evaluation

The bad example. To answer the when question uses local time, and not a standardized time demonstrating a bad clock. It does answer who, however the content used is bad business logic, as it shows unencrypted personal information, the user's email address. If a hacker got a hold of these logs, the hacker now has a list of every signed in user email address. The structure of the log for where is bad, as while it gives a URL for the page being accessed. If there are more applications that the SIEM pulls from, we would not know which application the log came from and if what wanted the controller or any of the code we would have to click on the URL to found out where the event happened, which is time-consuming processes as that would a human would have to do it manually instead of the SIEM doing it automatically. It doesn't answer what at all as the description is too vague and doesn't gives a full picture of what happened. Then for which, it answers it vague as it tells me what model in the database is being accessed but not how many.

The good example. In contrast, the good example clock is good as it uses a standardized time to answer the when question. It answers who with a unique ID, and the user's IP address. The where, if we had multiple applications being feed into the same SIEM,

is easy identifiable and shows in the application where it happened in the code. It answers what the user did, with what page it accessed, and if it succeeds. It answered which with what model was accessed in the database and how many were accessed.

5 Conclusions

This paper showed the security vulnerability of where logging is the practice of recording events and information, to use for security or as a timeline of events. Monitoring is a security practice of using the logs collected, compiling them, then analyzing them to notice trends then taking a response to abnormal events.

Event types, operational logs, and security logs. Where events are actions that create an individual log, how to monitor with SIEM, security information and event management, as the automated tool to collect, aggregate and analyze large volumes of logs and data and observability. Where it is using data collected to discover vulnerabilities in the application and produce responses to counter the vulnerabilities.

We then explained how logs are stored, the file system on the local hard drive or a database system like SQL. They can also be stored in cloud services such as in [2] with AWS CloudWatch.

log retention, how long a log should be kept in the database and how they are disposed of safely.

Then we explain what is considered insufficient and how to test and why it matters that this should be fixed. We went over what is good logging and monitoring, then we gave a short code demo with ruby on rails with the code and instructions on how to run it on github. Then we gave an analysis of the demo and explained in detail why they are good and bad.

6 Future works

If I were to do this project again, I would like to talk more about observability as being able to use data from the logs to discover vulnerabilities is really cool. As I believe you can test the middleware software, and some of the user's software using the data collected to discover vulnerabilities.

So, observability is to take the data in your logs and then test and verify to found cracks in the application so you can respond to them with your monitoring and that the trends that appear in the logs can be caught in a timely manner.

I would also like to change the severity name to type as that is a more apt term for it.

I would have also like to do a SIEM, having 2 maybe 3 applications and have them feed their logs into that SIEM as I don't have in my requirements to do or show bad and good monitoring. Just bad and good logging. To test alerts and demonstrate good alert business sense and then bad alert business sense.

I would also like to add the automated responses that a SIEM would have to do and play around with AI to try to brute force the applications I make and then block the IP address for their response. I would have also tried to show more logs, as in the demo I

have only showed one type of log and also do all of the automated monitoring like user monitoring and synthetic monitoring.

7 References

1. Logging - OWASP Cheat Sheet Series, last accessed 2026/04/16
2. AWS Prescriptive Guidance - Logging and monitoring guide for application owners, last accessed 2026/04/16
3. A09 Security Logging and Monitoring Failures - OWASP Top 10:2021, last accessed 2026/04/16
4. What Is SIEM? | Microsoft Security, last accessed 2026/04/17
5. The Strategic Importance of Observability | Cribl, last accessed 2026/04/17
6. Nist 800-171 Log Retention: Cybersecurity Best Practices, last accessed 2026/04/17